

DataCORE

Robustness, Interactive Visualization & Figure Captions

Afternoon: DataCORE

1. Final Analysis Cleanup & Robustness (1:00–2:00)
2. Interactive Visualization — Plotly (2:00–2:45)
3. Scientific Storytelling — Figure Captions (2:45–3:00)

*Your last chance to make the analysis **more convincing**: stress-test it, make one chart worth exploring, and caption it so it speaks for itself.*

1:00 – 2:00 · Final Analysis

What is a robustness check?

A finding you can only get **one way** is fragile. A robustness check asks:

1

Re-run your main analysis on a **subset** of the data — one year, one region, one group.

2

Does the result hold? Same direction, similar size?

3

Report what you find — either way it makes you stronger.



Holds? Your finding is more convincing.



Breaks? That's not a failure — it's a **limitation** to discuss honestly.

Robustness: re-run on one subgroup

Python

R

```
full = smf.ols("hourly_wage ~ experience_years", data=wages).fit()    # main analysis
urban = smf.ols("hourly_wage ~ experience_years",
               data=wages[wages.region == "Urban"]).fit()            # subset: Urban only
print(f"experience coef (all regions) = {full.params['experience_years']:.3f}")
```

experience coef (all regions) = 0.433

```
print(f"experience coef (Urban only) = {urban.params['experience_years']:.3f}")
```

experience coef (Urban only) = 0.508

 The Urban-only slope points the **same direction** and is close in size – the experience–wage link **survives** the subset. That sentence belongs in your Discussion.

+ Add one control – watch the coefficient move

Python R

```
base = smf.ols("hourly_wage ~ experience_years + C(gender)",  
              data=wages).fit()                                # baseline model  
plus = smf.ols("hourly_wage ~ experience_years + C(gender) + C(education)",  
              data=wages).fit()                                # + education control  
print(f"experience coef (baseline)          = {base.params['experience_years']:.3f}")
```

experience coef (baseline) = 0.426

```
print(f"experience coef (+ education)       = {plus.params['experience_years']:.3f}")
```

experience coef (+ education) = 0.407

 Adding **education** shifts the experience coefficient by about **4%**. **Interpret it:** if the coefficient barely moves, experience matters *independently* of schooling; if it drops a lot, part of what looked like an "experience effect" was really education.

AI limitation exercise

Ask ChatGPT:

"What are three limitations of a study that uses this method and this dataset?"

Paste in your method (e.g. OLS regression) and a one-line description of your data. **Take notes on all three.**

Then sort them yourself:

- Which limitations **actually apply** to your study?
- Which does the AI **get wrong** or invent?
- Which is the **most serious** for your claim?

 The survivors go straight into your **Discussion** section.

01:30

2:00 – 2:45 · Interactive Visualization

Why interactive charts?

A static chart shows **one view**. An interactive chart lets the reader **explore**:

- **Hover** to read exact values
- **Zoom** into a dense region
- **Filter** groups on and off

You'll use one library: **Plotly Express**, imported as `px`.

 These show up in **policy briefs, science journalism, and academic supplemental materials** — anywhere the audience wants to poke at the data themselves.

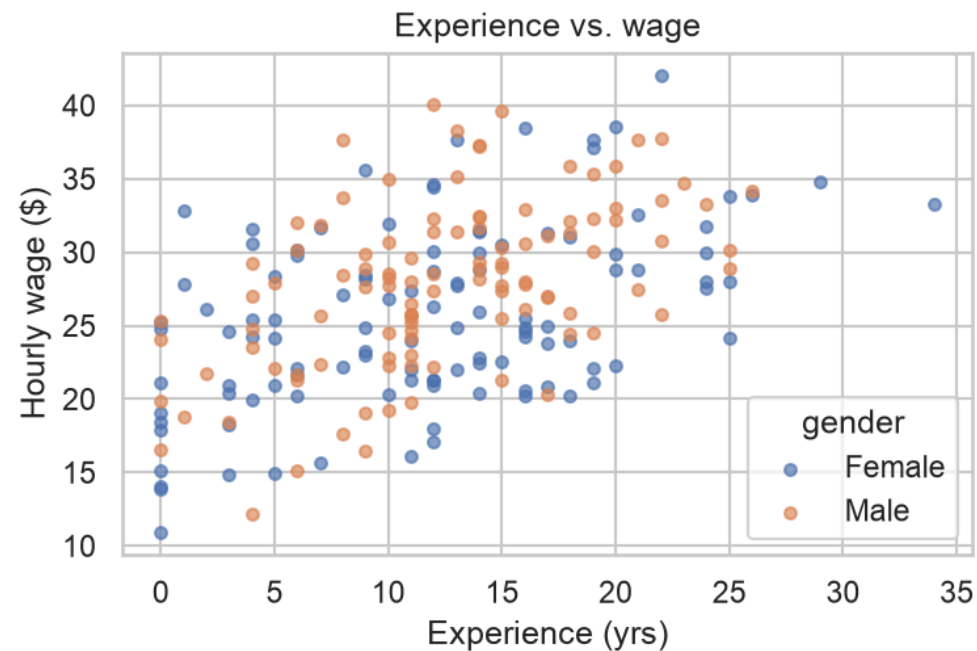
px.scatter() with hover details

Point-level detail appears on **hover** via

hover_data:

```
import plotly.express as px
fig = px.scatter(
    wages, x="experience_years", y="hourly_wage",
    color="gender",          # color by group
    hover_data=["education", "region"], # hover info
    labels={"experience_years": "Experience (yrs)",
            "hourly_wage": "Hourly wage ($)"},
    title="Experience vs. wage")
fig.show()
```

 Hovering any point reveals its education and region.

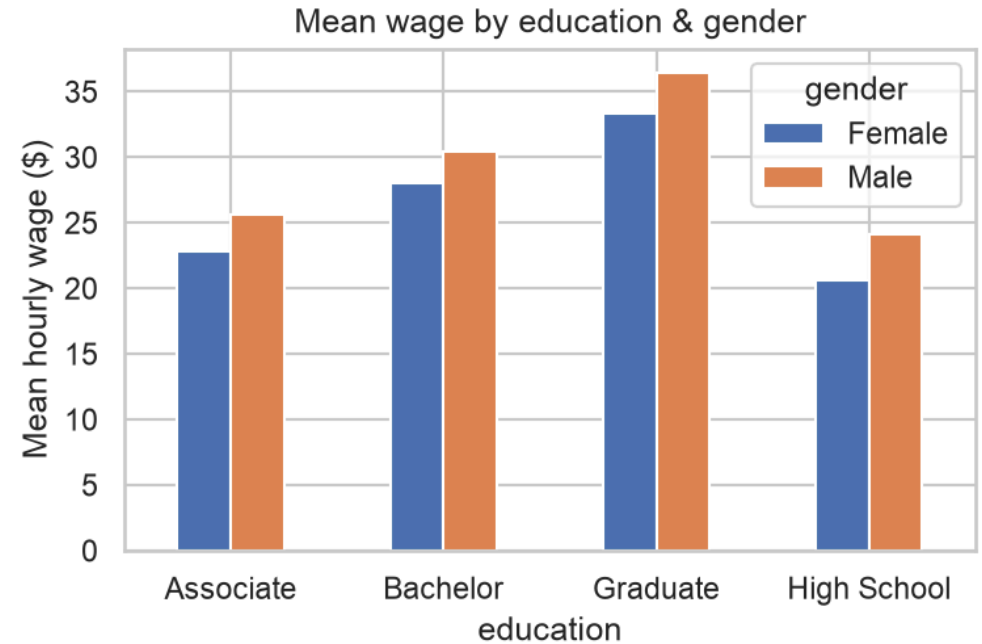


Static preview — the real `px` chart is zoomable & hoverable.

px.bar() – grouped by color

The `color` argument splits each bar into groups:

```
means = (wages.groupby(["education", "gender"])
          ["hourly_wage"].mean().reset_index())
fig = px.bar(
    means, x="education", y="hourly_wage",
    color="gender",          # grouped bars
    barmode="group",
    labels={"hourly_wage": "Mean hourly wage ($)"},
    title="Mean wage by education & gender")
fig.show()
```



Static preview – in Plotly, click a legend entry to filter a group.

Your deliverable – and how to export it



RcF

Create **one Plotly chart** of your main finding.



RcS

Create an **interactive scatter plot with a regression line** (`px.scatter(..., trendline="ols")`). Export as **HTML** for your portfolio.

Export the interactive chart as a standalone file:

```
# export as a standalone, still-interactive file  
fig.write_html("main_finding.html")
```



An **HTML** export keeps it interactive – a PNG would freeze it. Add the HTML to your portfolio and supplemental materials.

45:00

2:45 – 3:00 · Scientific Storytelling

A caption that works without you

Every chart in your Summit talk needs a caption that stands **alone** — three sentences:

- 1 **What the chart shows** — the variables and chart type.
- 2 **The key finding or pattern** — what the viewer should notice.
- 3 **The sample and data source** — who/what the data is, and where it came from.

“*This scatter plot shows hourly wage against years of experience for 220 workers. Wages rise with experience, roughly \$1 per additional year. Data come from the 2023 wage survey.*”

 **Read it aloud.** If a stranger could understand the chart from the caption alone, it's ready.

15:00

